

Machete de comandos · Clase 2

Preparando nuestro entorno de desarrollo · Seminario de Actualización · IFTS N.º 18

Guardá este machete. Todos los comandos de la clase están acá, en el mismo orden en que los usamos. Elegí la columna de tu sistema operativo.

1. Verificar qué tengo instalado

Windows (PowerShell)	macOS / Linux (Terminal)
<pre>python --version py --version py -0 # tus versiones de Python code --version git --version</pre>	<pre>python3 --version code --version git --version</pre>

Si alguno de estos comandos «no se reconoce», mirá la sección 6.

2. Configurar Git (una sola vez en la vida)

Windows (PowerShell)	macOS / Linux (Terminal)
<pre>git config --global user.name "Nombre Apellido" git config --global user.email "tu@correo.com" git config --global init.defaultBranch main git config --global --list</pre>	<pre>git config --global user.name "Nombre Apellido" git config --global user.email "tu@correo.com" git config --global init.defaultBranch main git config --global --list</pre>

Sin esto, Git no te deja hacer el primer commit.

3. Crear el proyecto y su entorno virtual

Primero creá la carpeta proyecto_integrador y abrila en VS Code (Archivo → Abrir carpeta). Después abrí la terminal integrada con Ctrl + ñ.

Windows (PowerShell)	macOS / Linux (Terminal)
<pre># ANTES de crear el entorno: python -c "import sys; print(sys.executable)" # Crear y activar: py -m venv .env .venv\Scripts\activate # DESPUÉS, el mismo comando: python -c "import sys; print(sys.executable)"</pre>	<pre># ANTES de crear el entorno: python3 -c "import sys; print(sys.executable)" # Crear y activar: python3 -m venv .env source .venv/bin/activate # DESPUÉS, el mismo comando: python -c "import sys; print(sys.executable)"</pre>

Cuando el entorno está activo, la terminal muestra (.venv) al principio de la línea.

Para salir del entorno, en los dos sistemas: deactivate

4. Instalar y registrar dependencias

Windows (PowerShell)	macOS / Linux (Terminal)
<pre>pip install requests pip list # Guardar la lista de dependencias: pip freeze > requirements.txt</pre>	<pre>pip install requests pip list # Guardar la lista de dependencias: pip freeze > requirements.txt</pre>

Windows (PowerShell)	macOS / Linux (Terminal)
# Reconstruir el entorno en otra máquina: pip install -r requirements.txt	# Reconstruir el entorno en otra máquina: pip install -r requirements.txt

5. Git: primer repositorio local

El orden importa: creamos el .gitignore ANTES del primer commit, para no guardar el .venv en el historial.

Windows (PowerShell)	macOS / Linux (Terminal)
git init git status -u # -u muestra TODOS los archivos # Crear .gitignore con este contenido: # .venv/ # __pycache__/ git status -u # miramos la diferencia git add . git commit -m "Estructura inicial del proyecto" git log --oneline	git init git status -u # Crear .gitignore con este contenido: # .venv/ # __pycache__/ git status -u git add . git commit -m "Estructura inicial del proyecto" git log --oneline

6. Si algo falla

Qué ves	Qué hacer
«python no se reconoce como un comando interno o externo» «command not found: python»	Cerrá y volvé a abrir la terminal (y VS Code entero). Si sigue igual, reinstalá Python tildando «Add python.exe to PATH». En macOS y Linux el comando es python3.
Al escribir python se abre la Microsoft Store	Configuración → Aplicaciones → Alias de ejecución de aplicaciones → apagá python.exe y python3.exe. Mientras tanto usá py.
«No se puede cargar el archivo Activate.ps1 porque la ejecución de scripts está deshabilitada en este sistema»	Ejecutá el comando de abajo y confirmá con S. Es seguro y solo afecta a tu usuario.
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned	Alternativa rápida: cambiar la terminal a Command Prompt y activar con .venv\Scripts\activate.bat
«Please tell me who you are» al hacer commit	Te falta la sección 2 de esta hoja: los dos comandos de git config.
git status no me muestra los archivos del .venv	Git colapsa las carpetas sin seguimiento. Usá git status -u para verlas todas.
VS Code ejecuta un Python distinto del que activé	Ctrl + Shift + P (Cmd en Mac) → escribí «Python: Select Interpreter» → elegí el que dice ('.venv': venv). Se ve confirmado en la barra inferior.
No encuentro la terminal en VS Code	Ctrl + ñ, o menú Terminal → Nueva terminal.
pip me da error de red o de proxy	Probá con el celular como punto de acceso. Si no funciona, avisá por el chat: seguimos en dupla y lo completás después con la grabación.

7. Palabras de hoy

Estas son las que van a aparecer esta noche. El glosario completo de la clase está en el aula virtual.

Término	Qué significa
intérprete (interpreter)	El programa que ejecuta tu código Python. Podés tener varios instalados.
entorno virtual (virtual environment)	Una carpeta con su propio Python y sus propias librerías, aislada del resto de la computadora.
dependencia (dependency)	Una librería que tu proyecto necesita para funcionar.
repositorio (repository)	La carpeta de tu proyecto con su historial de versiones adentro.
commit	Una foto guardada del estado del proyecto en un momento dado, con un mensaje que la describe.
untracked / staged / committed	Sin seguimiento / preparado para guardar / ya guardado en el historial. Son los tres estados que muestra git status.
Git vs GitHub	Git es el sistema de control de versiones y funciona en tu computadora. GitHub es un servicio en internet que aloja repositorios. Hoy usamos solo Git.

8. Cómo sé que terminé

Tu carpeta debería verse así:

```
proyecto_integrador/
├── .venv/           ← NO se versiona
├── .gitignore       ← SÍ se versiona
├── README.md        ← SÍ se versiona
├── requirements.txt ← SÍ se versiona
└── app.py           ← SÍ se versiona
```

Y para el ticket de salida, pegá en el foro el resultado de estos dos comandos:

```
python -c "import sys; print(sys.executable)"
git log --oneline
```

La actividad no lleva nota. Sirve para que arranquemos la Clase 3 con todo funcionando.